class searches for the specified tag with the specified property. In this instance, it will locate every *div* with the *class* entry content. Although we have all of the website's information, you can see that all the elements other than the <*a*> have been eliminated.

```
import requests
from bs4 import BeautifulSoup

# Making a GET request
r = requests.get('https://flask-bulma-css.appseed.us/')

# Parsing the HTML
htmlobj = BeautifulSoup(r.content, 'html.parser')

s = htmlobj.find('div', class_="sidebar")

content = s.find_all("a")

print(content)
```

The output is:

```
[<a class="sidebar-close" href="javascript:void(0);"><i
data-feather="x"></i></a>, <a href="#"><span class="fafa-
info-circle"></span>About</a>,<a href="https://github.com/
app-generator/flask-bulma-css" target="_blank">Sources</a>, <a
href="https://blog.appseed.us/bulmaplay-built-wit
h-flask-and-bulma-css/" target="_blank">Blog Article</a>,
<a href="#"><span class="fa fa-cog"></span>Support</a>, <a
href="https://discord.gg/f

ZC6hup" target="_blank">Discord</a>, <a href="https://
appseed.us/support" target="_blank">AppSeed</a>]
```

Identification of HTML elements

In the example above, we identified the components using the class name; now, let's look at how to identify items using their IDs. Let's begin this process by parsing the page's *cloned-navbar-menu* content. Examine the website to determine the tag of the '*cloned-navbar-menu*'. Then, finally, search all the *path* tags under the tag <*a*>.

```
import requests
from bs4 import BeautifulSoup

# Making a GET request
r = requests.get('https://flask-bulma-css.appseed.us/')

# Parsing the HTML
htmlObj = BeautifulSoup(r.content, 'html.parser')

# Finding by id
s = htmlObj.find('div', id="cloned-navbar-menu")

# Getting the "cloned-navbar-menu"
a = s.find('a' , class_="navbar-item is-hidden-mobile")

# All the paths under the above <a>
content = a.find_all('path')
print(content)
```

```
[<path class="path1" d="M 300 400 L 700 400 C 900 400 900 750
600 850 A 400 400 0 0 1 200 200 L 800 800"></p

ath>, <path class="path2" d="M 300 500 L 700 500"></path>,
<path class="path3" d="M 700 600 L 300 600 C 100
600 100 200 400 150 A 400 380 0 1 1 200 800 L 800 200"></
path>]
```

Iterating over a list of multiple URLs

To scrape many sites, each of the URLs need to be scraped individually, and it is required to hand-code a script for each such web page.

Simply create a list of these URLs and loop over it. It is not required to create code for each page to extract the titles of those pages; instead, just iterate over the list's components. You may achieve that by following the example code provided here.

```
import requests
from bs4 import BeautifulSoup as bs

URL = ['https://www.researchgate.net', 'https://www.linkedin.
com']

for url in range(1,2):
    req = requests.get(URL[url])
    htmlObj = bs(req.text, 'html.parser')

    titles = htmlObj.find_all('titles',attrs={'class','head'})
    if titles:
        for i in range(1,7):
            if url+1 > 1:
                print(titles[i].text)
            else:
                print(titles[i].text)
```

Storing web contents into a CSV file

After scraping the web pages, one can store the elements into a CSV file. Let's consider the storing of a tag-value dictionary list in a CSV file. The dictionary elements will be written to a CSV file using the CSV module. Check out the example below for a better understanding.

Continued to page...82